

Tel-Hai
University of Kiryat Shmona On the rise

K-Nearest Neighbors & Supervised Learning

Dr. Marcelo Feighelstein

Course Roadmap

What We'll Cover Today

01

Supervised Learning

What it is, how it works, and why it matters

02

Train / Test Split

How we evaluate a model fairly — and why it's critical

03

KNN Algorithm

Learning by proximity — intuition first, math second

04

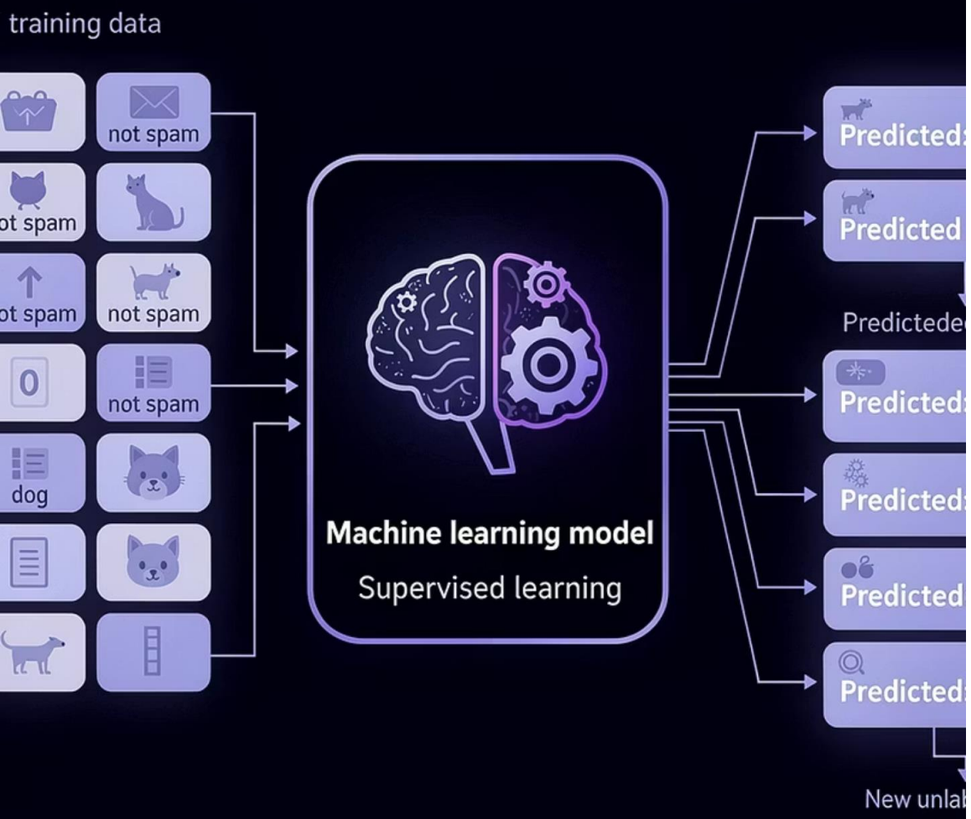
Feature Scaling

Why distance-based models demand equal treatment of features

05

Real Datasets

Applying everything to Iris and Titanic



What Is Supervised Learning?

Supervised learning trains a model on **labeled examples**, so it can learn the relationship between **features** and **labels**. It then uses that relationship to predict new data.

Medical Diagnosis

Input: age, blood pressure, test results → Output: disease / no disease.

Spam Detection

Input: email text, sender, links → Output: spam / not spam.

Flower Classification

Input: petal size, sepal size → Output: species name.

> “Supervised” means the model learns from data with correct answers.

Rules vs. Learning: Two Approaches

Before machine learning, engineers wrote explicit rules to make decisions. ML takes a different approach — it **learns the rules automatically** from data. Here's how they compare:

Rule-Based Systems

- Hand-crafted if-else logic
- Requires domain expertise
- Brittle: breaks on edge cases
- Hard to maintain and scale

Example: IF petal_length > 5 AND petal_width > 1.8 THEN Virginica

Machine Learning Models

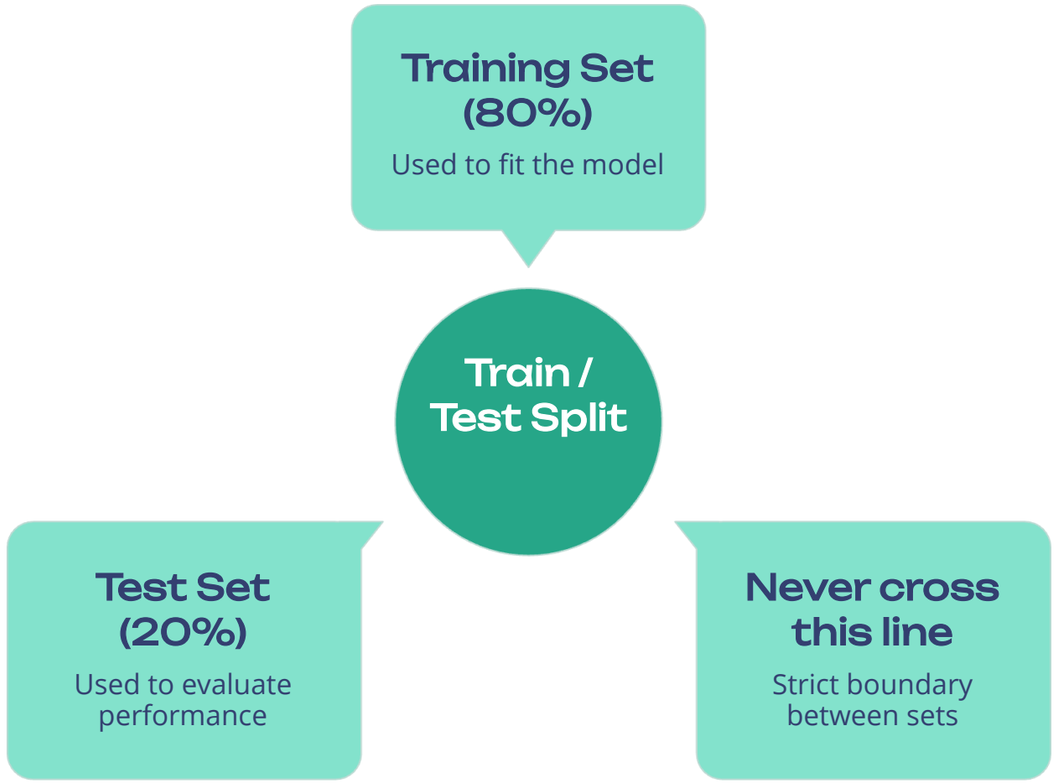
- Learns patterns directly from data
- Generalizes to unseen examples
- Adapts as data grows
- Discovers complex boundaries automatically

Example: Train KNN on 150 examples → model finds the boundary itself

🤔 **Think about it:** Could you write rules to classify a flower just from measurements? How many rules would you need?

The Train / Test Split

To measure how well a model *generalizes*, we evaluate it on data it has **never seen** during training.



🌱 Training Set (~80%)

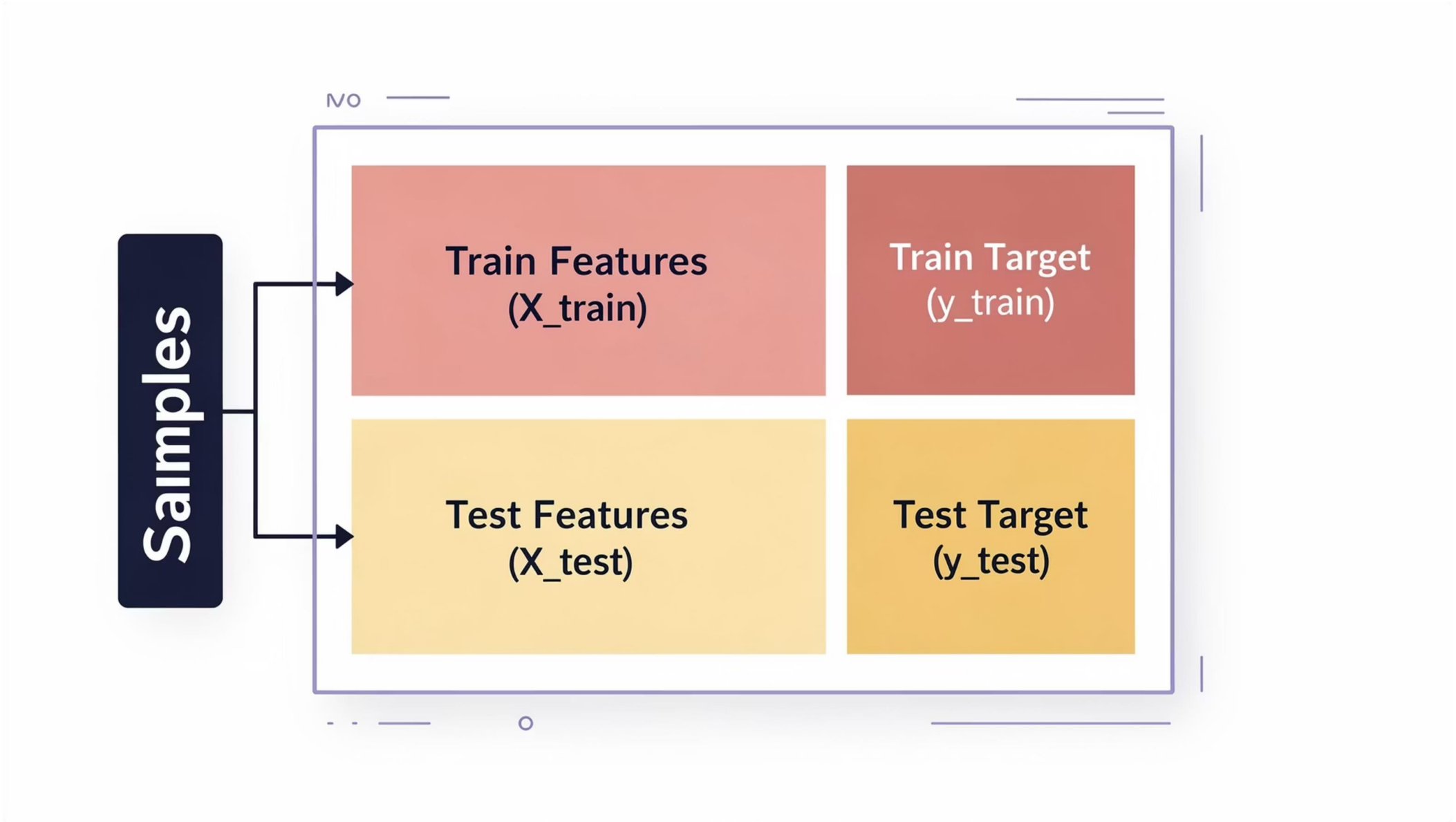
The model **learns** from this data.

🧪 Test Set (~20%)

We use this **only once** at the end.

⚠ Never train on the test set.

Data Split for Training & Testing

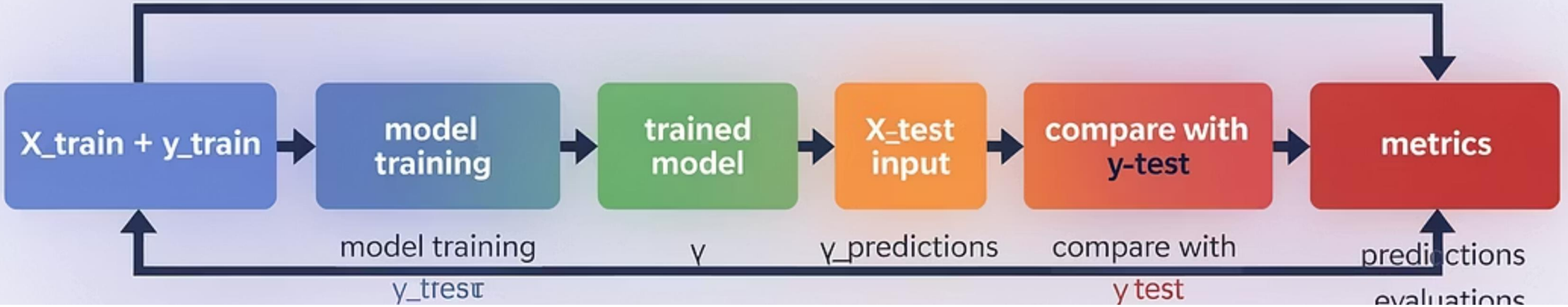


Training

X_train (features) + **y_train** (labels) are **both** used together to fit the model.
The model learns patterns from the input features and the correct answers at the same time.

Testing

Only **X_test** (features) is fed to the model → the model outputs predictions.
Those predictions are then compared against **y_test** to evaluate performance.



From Training to Prediction

🏋️ Model Training

X_train + y_train → model fits the data

🔮 Prediction

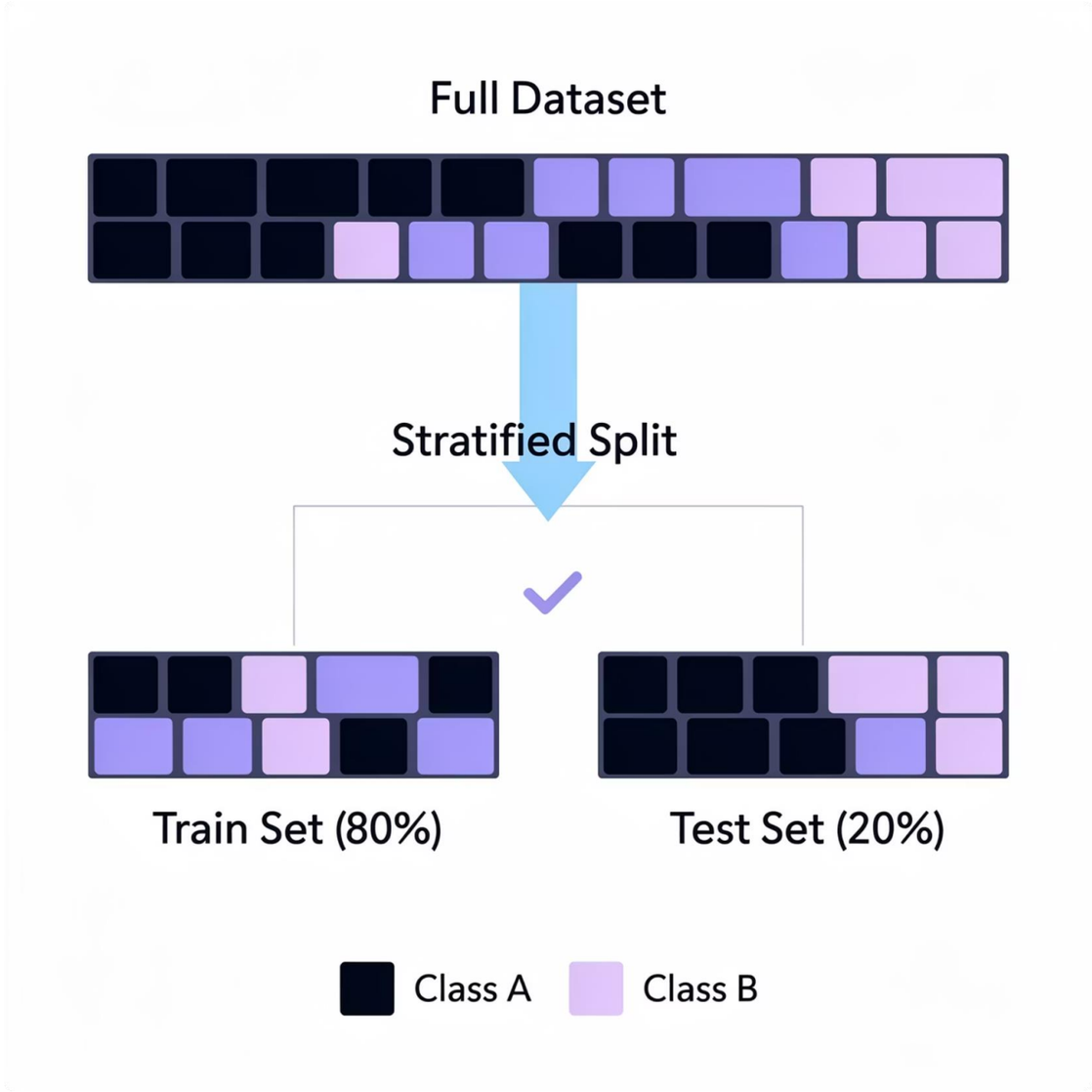
X_test only → trained model outputs ŷ

📊 Evaluation

ŷ vs y_test → accuracy & metrics computed

⚠️ y_test is never shown to the model — it is only used AFTER predictions are made to measure performance.

Stratification



Why it matters

A random split might accidentally put all samples of one class in training only. Stratification ensures both Train and Test sets preserve the same class ratio.

Use `stratify=y` in `train_test_split`



Reproducibility & Random Seed

Splitting a dataset randomly means results change each run. Setting a random seed (e.g., random_state=42) freezes the randomness so your results are reproducible — essential for scientific work and debugging.

A seed is a starting number given to a random number generator. Using the same seed always produces the same sequence of random numbers — so the split is identical every time you run the code.

Same seed → same split → same results, every time.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42, stratify=y)
```

Meet the Iris Dataset

The **Iris dataset** is one of the most famous datasets in machine learning — a perfect sandbox for classification. It contains **150 labeled flower samples** across 3 species.



Setosa

50 samples — easiest to separate. Small petals, very distinct.



Versicolor

50 samples — moderate overlap with Virginica. The tricky middle class.



Virginica

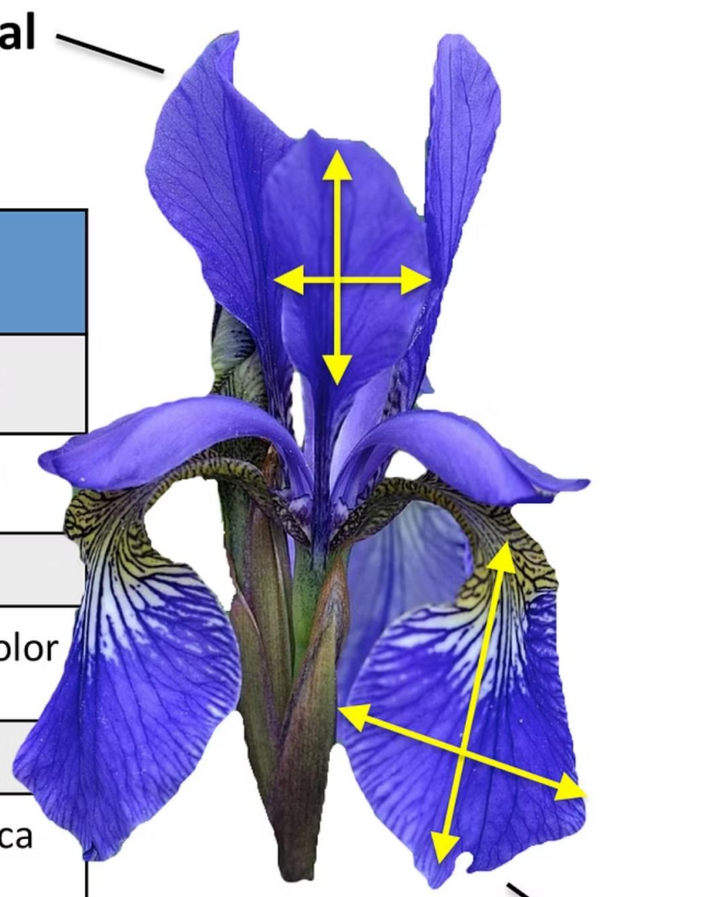
50 samples — largest petals. Sometimes confused with Versicolor.

The 4 features: **sepal length**, **sepal width**, **petal length**, **petal width** — all measured in centimeters.

Iris Anatomy: Petals & Sepals

Samples
(instances, observations)

	Sepal length	Sepal width	Petal length	Petal width	Class label
1	5.1	3.5	1.4	0.2	Setosa
2	4.9	3.0	1.4	0.2	Setosa
...					
50	6.4	3.5	4.5	1.2	Versicolor
...					
150	5.9	3.0	5.0	1.8	Virginica



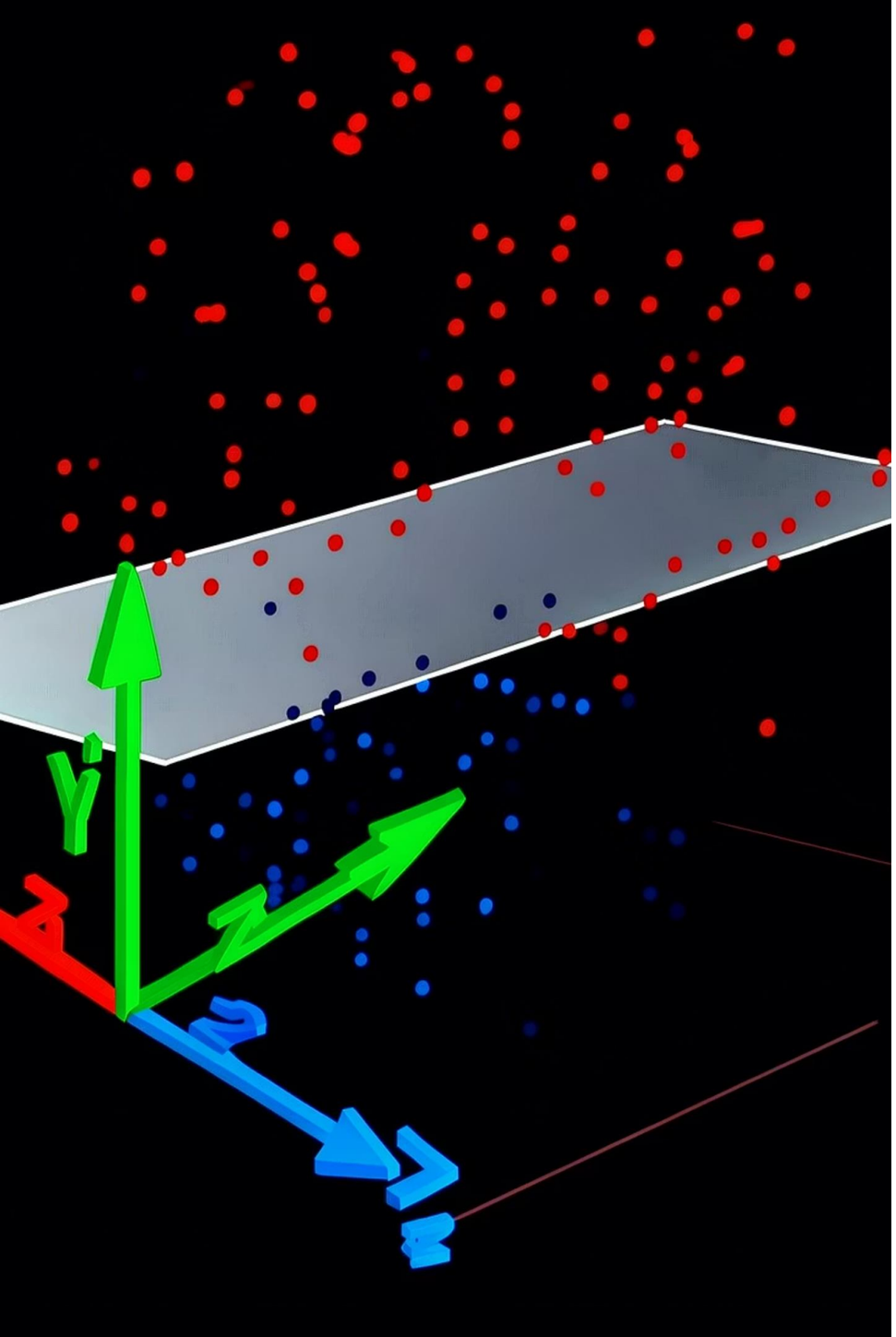
Iris Dataset Samples

Here's a glimpse into the structure of the Iris dataset. Notice how the measurements differentiate the species.

Sample ID	Sepal Length (cm)	Sepal Width (cm)	Petal Length (cm)	Petal Width (cm)	Species
0	5.1	3.5	1.4	0.2	Iris Setosa
1	4.9	3.0	1.4	0.2	Iris Setosa
50	5.9	3.2	4.8	1.8	Iris Versicolor
51	5.7	2.8	4.1	1.3	Iris Versicolor
100	6.3	3.3	6.0	2.5	Iris Virginica
101	6.2	3.4	5.4	2.3	Iris Virginica

Sample ID is the original dataset index and is not used as a feature for classification.

The dataset contains exactly 50 samples per flower type — 50 Setosa, 50 Versicolor, and 50 Virginica — for a total of 150 samples.



Data as Points in Space

Every row in your dataset is a **point in multidimensional space**. Each feature adds one dimension. This geometric view is the foundation for understanding distance-based algorithms.

1

1D

1 feature → points on a **line**

2

2D

2 features → points on a **plane**

3

3D

3 features → points in **space**

4

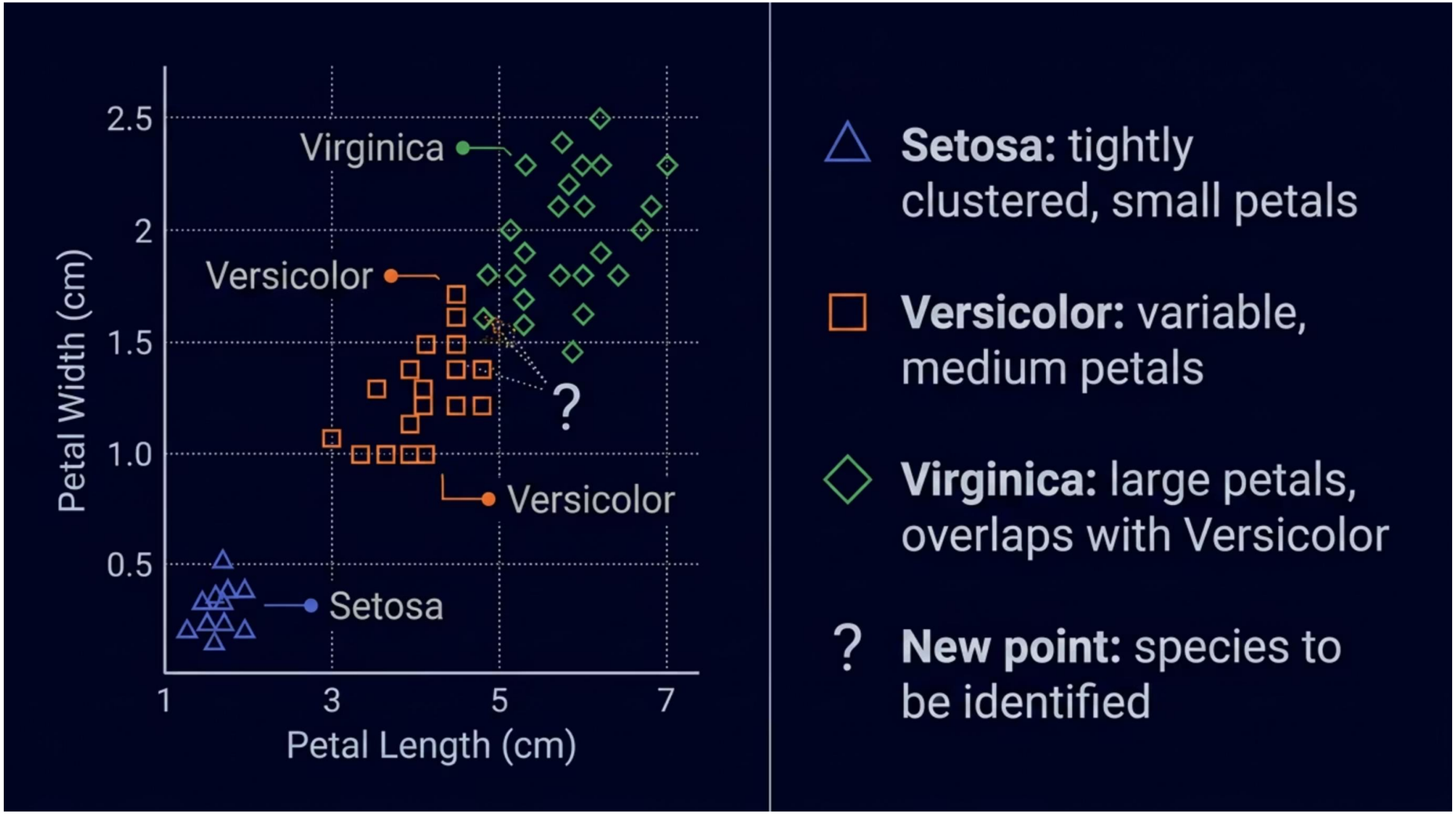
ND

N features → abstract **hyperspace**

Classification becomes: **draw a boundary (hyperplane) that separates different classes** in this space. With KNN, we don't draw an explicit boundary — we let the neighbors decide.

Visualizing Iris in 2D

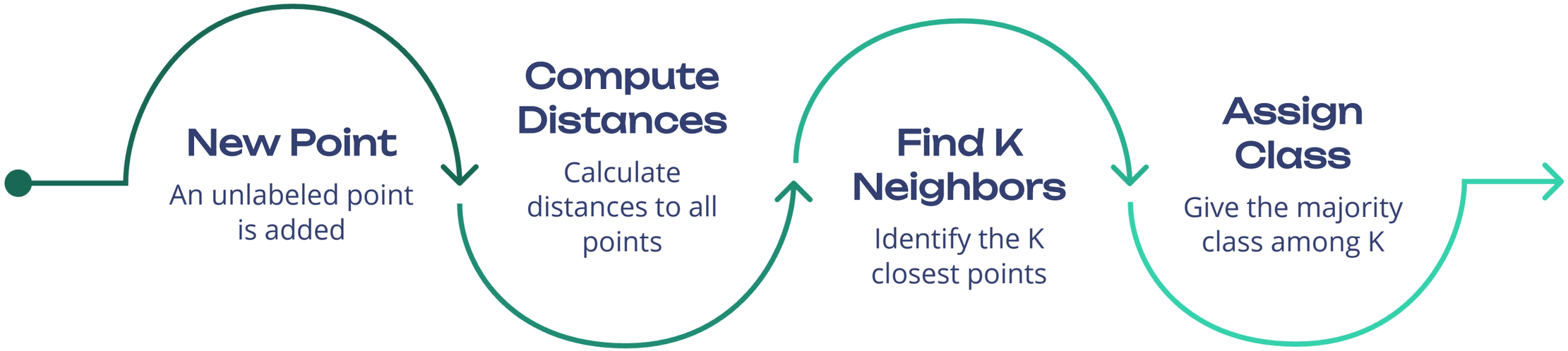
We can't easily see 4 dimensions, so let's **pick two features** — petal length and petal width — and plot each flower as a point. This 2D view reveals striking **natural clusters** in the data.



🤔 **Think about it:** If a new flower appears with petal length = 5.0 cm and petal width = 1.7 cm — which cluster does it fall near? How would *you* classify it?

KNN in Action: A Visual Walkthrough

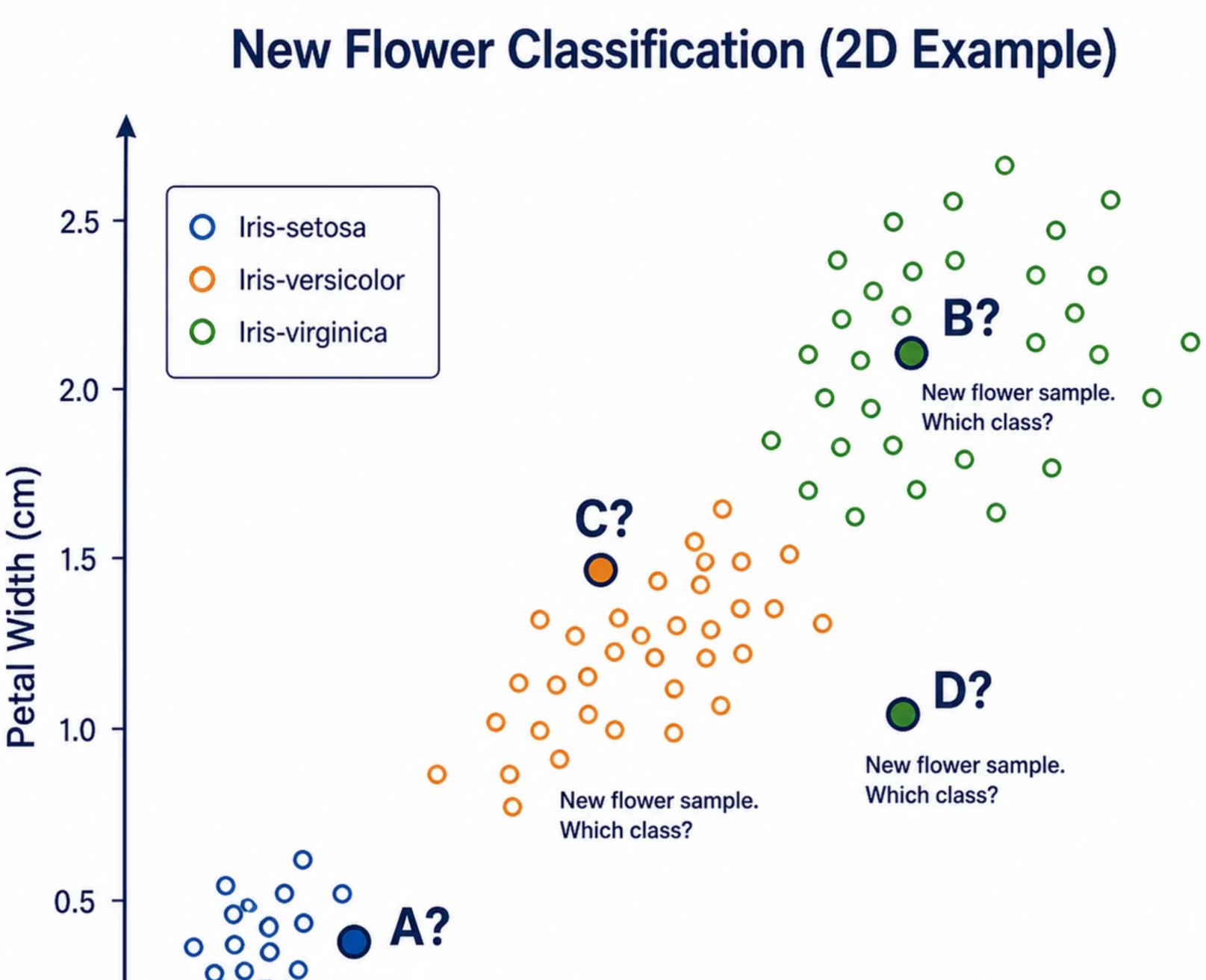
Let's visualize the K-Nearest Neighbors process step-by-step. A new, unlabeled data point enters the scene, seeking its identity among known neighbors.



This intuitive process allows the model to classify new observations based on the characteristics of its nearest companions, without needing to learn complex internal rules.

KNN in Action: Classifying New Points

Using our 2D visualization of the Iris dataset (Petal Length vs. Petal Width), let's apply the K-Nearest Neighbors principle to classify some new, unlabeled flower samples. Imagine K=3 for this exercise.



Choosing K: The Key Hyperparameter

K controls how many neighbors vote. It's a **hyperparameter** — you set it before training. The choice of K dramatically affects model behavior, and there's a fundamental tradeoff involved.

K = 1

Very sensitive. Classifies based on the single nearest neighbor. Memorizes training data — wiggly, complex boundary. High **variance**, low bias. Overfits noise.

K = 5

Balanced. Considers 5 neighbors before deciding. Smooths out noise while staying flexible. Often a good starting point for real problems.

K = 15+

Very smooth. Large neighborhood → simple, generalized boundary. High **bias**, low variance. May underfit if classes are close together.

ℹ️ 💡 Rule of thumb: Start with K = **√n** (square root of training samples) as a baseline, then tune using cross-validation.

⚠️ ⚠️ Hyperparameter tuning (choosing K) must be performed using the training set only — never the test set. The test set is reserved exclusively for final evaluation.

From 2D to 4D: The Full Iris Model

What Changes in 4D?

When we use all 4 Iris features, each point lives in a **4-dimensional space**. We can't visualize it directly — but mathematically, **distance still works exactly the same way**.

The Euclidean distance formula simply extends: instead of $\sqrt{((x_2-x_1)^2 + (y_2-y_1)^2)}$, we add terms for each extra dimension. The algorithm doesn't care how many dimensions there are.

More features → more information → potentially better separation between classes.

Key Insight

2D model

Uses petal length + width only. Visualizable, but ignores half the data.

4D model

Uses all features. Can't draw it — but achieves higher accuracy.

The principle

Distance in N dimensions = same math, more dimensions.

KNN in Python: The Full Workflow

Here's what a complete KNN workflow looks like in scikit-learn — from data loading to accuracy evaluation. Each step maps directly to the conceptual steps we discussed.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# 1. Load data
X, y = load_iris(return_X_y=True)

# 2. Split into train and test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 3. Create and train the model
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

# 4. Evaluate on test set
y_pred = knn.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Notice `stratify=y` — this ensures each class is proportionally represented in both train and test splits.

K-Tuning: Finding the Optimal K

Selecting the right 'K' value is crucial for KNN performance. Here, we loop through a range of K values, fit each model on `X_train`, and compare `train accuracy` to choose the best K. Once the optimal K is selected, we fit the final model one time and evaluate it on `X_test` as a final, unbiased check.

```
from sklearn.metrics import accuracy_score
from sklearn.neighbors import KNeighborsClassifier
import numpy as np

# Assuming X_train, X_test, y_train, y_test are already defined
k_range = range(1, 21)
k_scores = []

for k in k_range:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)
    y_pred_train = knn.predict(X_train)
    score = accuracy_score(y_train, y_pred_train)
    k_scores.append(score)

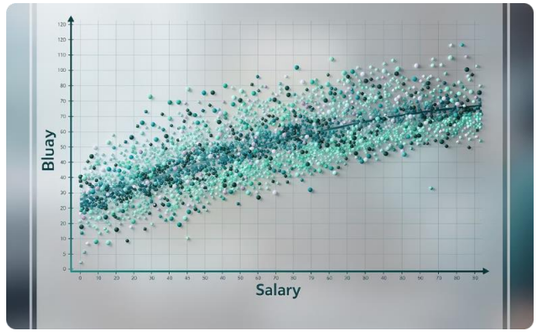
# Select best K based on train accuracy
optimal_k_index = np.argmax(k_scores)
optimal_k = list(k_range)[optimal_k_index]

print(f"Optimal K (by train accuracy): {optimal_k}")
print(f"Train accuracy: {k_scores[optimal_k_index]:.4f}")

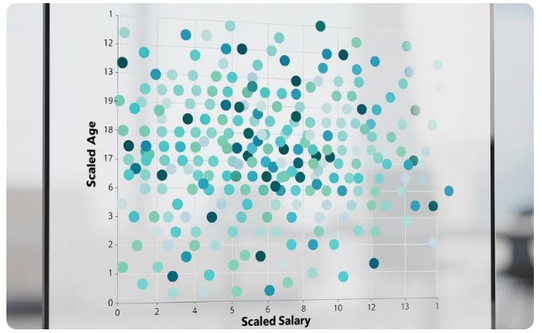
# Evaluate on test set using the best K
knn_best = KNeighborsClassifier(n_neighbors=optimal_k)
knn_best.fit(X_train, y_train)
y_pred_test = knn_best.predict(X_test)
print(f"Test accuracy: {accuracy_score(y_test, y_pred_test):.4f}")
```

Why Scaling Matters

KNN relies on **distance**. If one feature has values in the thousands and another in the single digits, the large-scale feature will **dominate the distance calculation** — even if it's not the most important feature.



Before Scaling



After Scaling

Without Scaling: Problem

Consider two features:

- **Age:** 25, 30, 35 → range ≈ 10
- **Salary:** 40,000 – 120,000 → range ≈ 80,000

Distance is almost entirely determined by salary. Age becomes invisible — even if it's a critical predictor.

With Scaling: Solution

After **Min-Max scaling**, both features map to [0, 1]:

- Age: 0.0 → 1.0
- Salary: 0.0 → 1.0

Now both features contribute **equally** to the distance calculation. The model can fairly weigh all dimensions.

💡 Why Iris doesn't need scaling: All 4 Iris features (sepal/petal length and width) are measured in the same unit (cm) and have similar ranges (~0.1–7.9 cm). So no single feature dominates the distance calculation — making scaling less critical. However, it's still good practice to always scale.

Min-Max Scaling: The Formula

The most common scaling technique for KNN is **Min-Max Normalization**. It squashes every feature into the range [0, 1] while preserving the relative ordering of values.

$$x_{\text{scaled}} = \frac{x - x_{\text{min}}}{x_{\text{max}} - x_{\text{min}}}$$

Intuition

- The minimum value becomes **0**
- The maximum value becomes **1**
- All other values scale proportionally in between
- The shape of the distribution is preserved

In Python

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

⚠ Always `fit` the scaler on training data only. Then `transform` the test data — never fit on test!

KNN on the Titanic Dataset

The Titanic dataset is a classic **binary classification** problem: predict survival (1) or death (0). It introduces more realistic challenges — missing values, mixed feature scales, and the need for careful preprocessing.



Age

Range: 0–80 years. Contains **missing values** that must be imputed before modeling.



Pclass

Values: 1, 2, 3. Small integers representing passenger class. Needs **encoding** for numeric features.



Fare

Range: $0–$512. **Hugely different scale** from age — will dominate KNN without scaling.



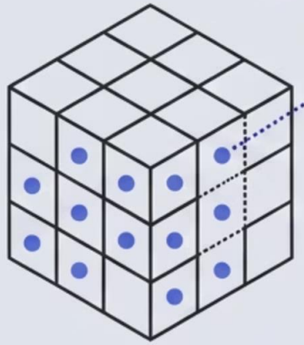
SibSp / Parch

Count of siblings/spouses and parents/children aboard. Small integers — still benefit from scaling.

Evaluating Your Model: Beyond Accuracy

Accuracy can be misleading with imbalanced classes; on Titanic, predicting “everyone dies” is already 62% accurate.

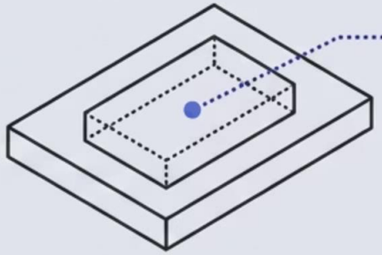
Accuracy



$$\frac{\text{Correct Predictions}}{\text{Total}}$$

Correct predictions / total.
Prefer for balanced classes.

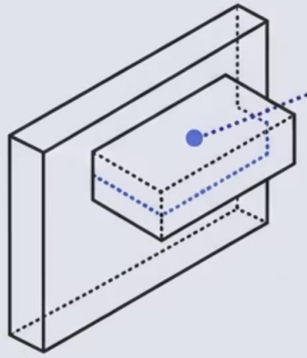
Precision



True Positives / Predicted Positives.

When you say "yes", how often right?
Prefer for low false positives.

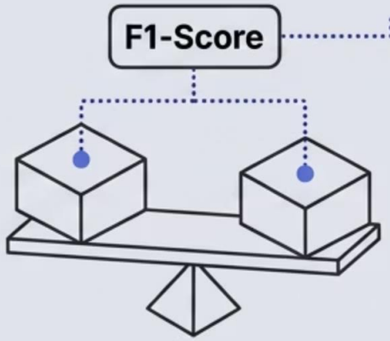
Recall



$$\frac{\text{True Positives /}}{\text{Actual Positives}}$$

How many real positives caught?
Prefer for low false negatives.

F1-Score



Harmonic mean of Precision & Recall

Harmonic mean of Precision and Recall.
Balances both.
Prefer for uneven class distribution.

Common Pitfalls to Avoid

🚫 Data Leakage

Fitting your scaler (or any preprocessor) on the **full dataset** before splitting leaks test information into training. Always split first, then fit the scaler on training data only.

🚫 Ignoring Class Imbalance

If one class has far more samples, the model may predict it exclusively. Use `stratify=y` when splitting and consider balanced metrics like F1.

🚫 Skipping Scaling

Running KNN on unscaled Titanic data gives misleadingly poor results. Always scale features before fitting any distance-based model.

🚫 Wrong K for the Dataset

K=1 overfits; K=100 underfits. Use cross-validation to find the optimal K — don't guess. Plot accuracy vs K to visualize the tradeoff.

Key Takeaways

Supervised Learning

Train on labeled data → predict labels for new inputs. Features in, prediction out.

Train / Test Split

Evaluate fairly on held-out data. Never let the model see test data during training.

KNN Algorithm

Find K nearest neighbors → majority vote wins. Distance drives everything.

Scaling Is Essential

Normalize features so no single dimension dominates distance calculations.

Visualize to Understand

2D plots build intuition — but models operate in full-dimensional space.

You now have the conceptual tools to understand, implement, and evaluate a KNN classifier on real-world data. Next step: experiment with different K values and datasets!



Thank You!

Go explore data — get your hands dirty, break things, and learn by doing.

Email

MarcelFei@telhai.ac.il

Office Hours

Per Request

Location

Building 6, Room 309, Eastern Campus